

Catalyst - Leverage Trading Protocol - High Level Technical Design Document

Table of content

- 1. Problem Statement
- 2. Proposed Solution
- 3. System Architecture
- 4. How the Solution Works
 - 4.1 Open Position
 - 4.2 Order Execution
 - 4.3 Close Position (Take Profit / Stop Loss)
 - 4.4 Risk Management : Liquidation
 - 4.5 User Monitoring
- 5. Onchain Design Overview
 - 5.1 Smart Contract Components
 - Protocol Configuration
 - Protocol Business Scripts
 - 5.2 UTXO & Token Model
 - 5.3 Onchain Data Structures (Datums)
- 6. Onchain Transaction Flows
 - 6.1 Create Leveraged Position
 - 6.2 Minswap Batching (Order Execution)
 - 6.3 Update Trading Loan
 - 6.4 Stop Loss / Take Profit
 - 6.5 Liquidate Trading Loan
 - 6.6 Collect Liquidated Trading Loan
- 7. Health Factor Calculation
- 8. Risk Management & Liquidation
- 9. Backward Compatibility

1. Problem Statement

Currently, the Cardano ecosystem offers perpetual trading but **does not support leverage trading with onchain limit orders**. This leaves traders without the ability to set automated entries, stop losses, or take profit targets when borrowing assets to trade through DEX liquidity. As a result, traders must monitor prices manually, limiting strategy diversity and discouraging professional traders from participating.

2. Proposed Solution

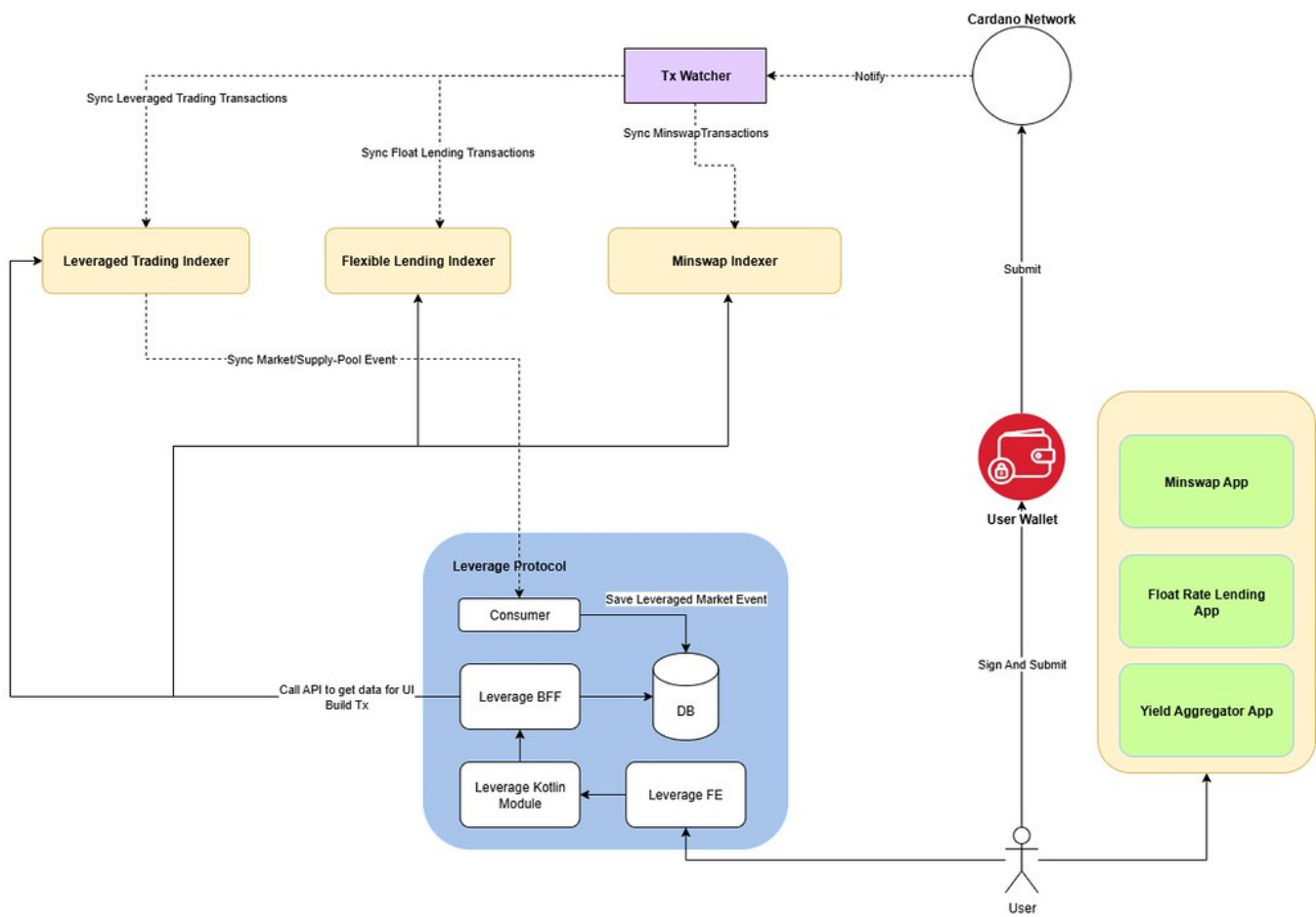
Danogo is developing **Cardano's first spot leverage trading protocol** by integrating:

- **Danogo Flexible Lending Pool** : for borrow/lend infrastructure (floating rate liquidity pools)
- **Minswap** : Cardano's largest DEX, providing batcher based market and limit order execution

This combination enables traders to:

- **Open and close leveraged positions** directly through Danogo Flexible Lending Pool and Minswap liquidity pools.
- **Place limit, stop loss, and take profit orders** stored onchain and executed automatically when price conditions are met.
- **Benefit from transparent liquidation flows**, where collateral is swapped onchain in a single transaction.

3. System Architecture

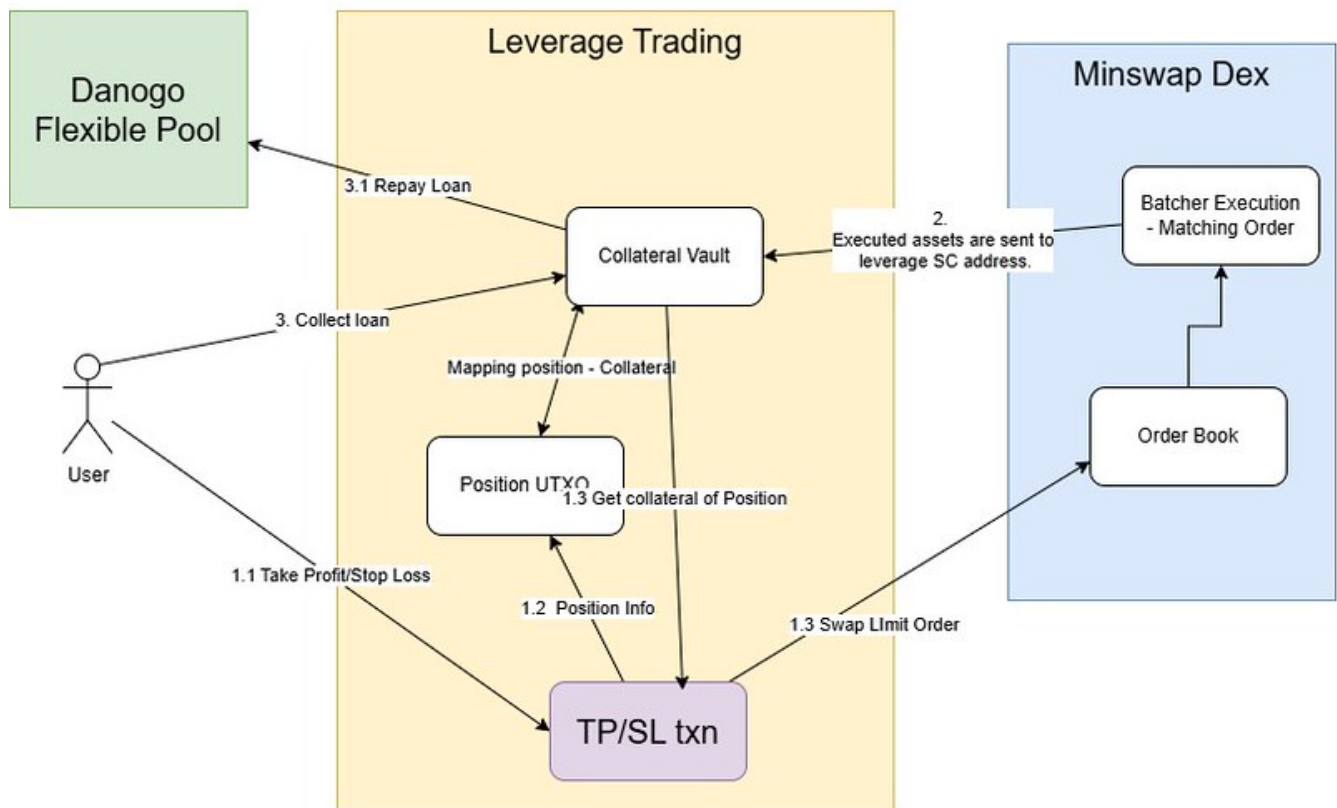
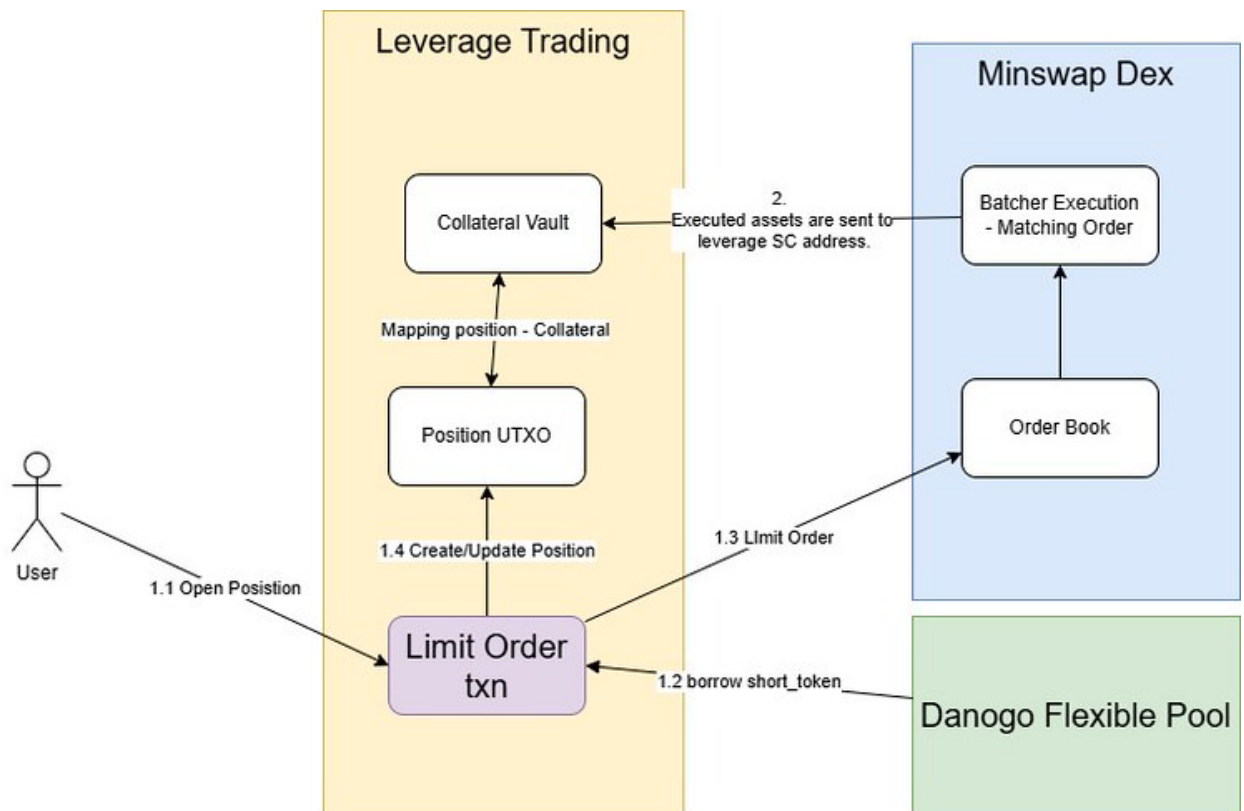


The system is composed of the following high level components:

	Component	Role
1	Cardano Network	Settlement layer for all onchain transactions
2	Tx Watcher	Listens to Cardano blocks and dispatches relevant transactions to indexers
3	Leveraged Trading Indexer	Indexes leverage position events (create, update, liquidate, collect)
4	Flexible Lending Indexer	Indexes lending pool events (supply, borrow, repay)
5	Minswap Indexer	Indexes Minswap swap order events (placed, matched, cancelled)
6	Consumer	Processes indexed events and persists market data to the database
7	Leverage BFF	Backend for Frontend API layer : serves UI data and builds unsigned transactions
8	Leverage Kotlin Module	Core business logic for transaction construction and validation
9	Leverage FE	Trader facing dashboard for monitoring positions, PnL, and placing orders
10	DB	Persistent storage for position state, market data, and indexed events
11	User Wallet	Signs and submits transactions to the Cardano Network

External integrations: **Minswap App**, **Float Rate Lending App**, **Yield Aggregator App**

4. How the Solution Works



4.1 Open Position

A trader opens a leveraged position using either a **market** or **limit** price:

1. The trader deposits collateral tokens and specifies leverage parameters (direction, leverage ratio, and limit price if applicable).
2. A single transaction is submitted to the Cardano Network that simultaneously:
 - **Borrows assets** from the Danogo Flexible Lending Pool.
 - Creates a **Minswap swap order** (market order uses current pool price; limit order uses the trader specified price).
 - Creates a **Position UTXO** (Loan) onchain, recording the debt amount, interest index, and order state.

4.2 Order Execution

The system relies on **Minswap's batcher mechanism** for order matching:

- Market orders execute at the current pool price.
- Limit orders remain in Minswap's onchain order book until the pool price meets the specified condition.
- Once matched, swapped tokens are sent back to the Leverage Protocol smart contract address, where they are held in a Vault UTXO linked to the corresponding Loan.

4.3 Close Position (Take Profit / Stop Loss)

Traders can set **take profit** or **stop loss** orders for fully or partially filled positions:

1. The trader specifies a target price and the amount of collateral to sell.
2. The protocol takes collateral from the Position UTXO and places a **swap limit order** on Minswap.
3. When the order is filled:
 - **Stop Loss** Swapped tokens are sent to the Pool to repay the borrowed amount; any shortfall is absorbed.
 - **Take Profit** Swapped tokens first repay the loan, then remaining profit is sent to the trader's wallet.
4. If the full position is closed, the Loan NFT is burned.

4.4 Risk Management : Liquidation

If market prices move against the trader and the position's **health factor (HF)** falls to or below **1**, the position enters a liquidatable state:

1. Any user (liquidator) can trigger liquidation by calling the protocol.
2. The protocol repays the loan using onchain mechanisms: swapping collateral through DEX protocols that support direct swaps (e.g., Splash, Danogo Concentrated), or accepting tokens directly from the liquidator to settle the debt in an onchain transaction to repay the loan.
3. The **liquidator receives a reward** (configured per pool) for maintaining protocol solvency.
4. A **Collect** step follows, where a bot aggregates liquidated UTXOs back into the Pool.

4.5 User Monitoring

Through the dApp dashboard, traders can monitor real time metrics: Entry price, Current PnL (Profit and Loss), Liquidation price, Active TP/SL orders, Health Factor.

This workflow delivers **fully automated leverage trading with conditional orders**, combining Danogo's lending infrastructure with Minswap's liquidity and batcher system.

5. Onchain Design Overview

5.1 Smart Contract Components

The onchain architecture consists of two core script groups:

Protocol Configuration

Script	Purpose
Protocol Config Script	Stores script key hashes enabling mutual script recognition. Immutable after deployment (no redeemer).

Protocol Business Scripts

Script	Purpose
Pool Param Script	Manages pool configuration (collateral tokens, liquidation thresholds, interest rates, fees). Supports CreatePool, UpdateParam, and ClosePool redeemers.
Pool Script	Holds pool liquidity (operator supplied tokens). Supports supply/withdraw (TopupWithdraw), loan operations (UpsertLoan), collecting liquidated positions, stop loss collection, rebalancing alternative tokens, fee withdrawal, and pool closure.
Loan Script	Manages individual trading positions (debt records, collateral custody). Supports creating/updating/closing loans (UpsertLoan), TP/SL order placement, liquidation, collection of liquidated loans, stop loss collection, and profit distribution.

5.2 UTXO & Token Model

Token	Minted By	Name Format	Purpose
Pool NFT (×2)	Pool Param	blake2b_224(out_ref)	Identifies Pool and Pool Param UTXOs
dToken	Pool	Same as Pool NFT name	Represents share of supplied liquidity
Loan NFT	Loan	blake2b_224(out_ref)	Identifies a Loan (position) UTXO
Loan Owner NFT	Loan	Loan NFT name + "OWN"	Proves ownership of a position
Vault NFT	Loan	Loan NFT name + "VLT"	Links a Minswap swap order (pre fill) or Vault UTXO (post fill) to its Loan
Collect FT	Loan	""	Marks a UTXO awaiting collection back into the Pool

5.3 Onchain Data Structures (Datums)

Loan Datum

```
LoanDatum {
  pool_nft_name      : AssetName  -- identifies the parent pool
  loan_amount        : Int        -- total borrowed amount
  initial_interest_index: Int      -- interest index at creation/last update
  is_exist_vault     : Bool       -- whether a linked Vault UTXO exists
}
```

Vault Datum

```
VaultDatum {
  received_address : Address
  -- Loan address   active collateral (counts toward HF)
  -- Pool address   stop-loss repayment (awaiting collection)
  -- User address   take-profit (awaiting distribution)
}
```

Pool Datum

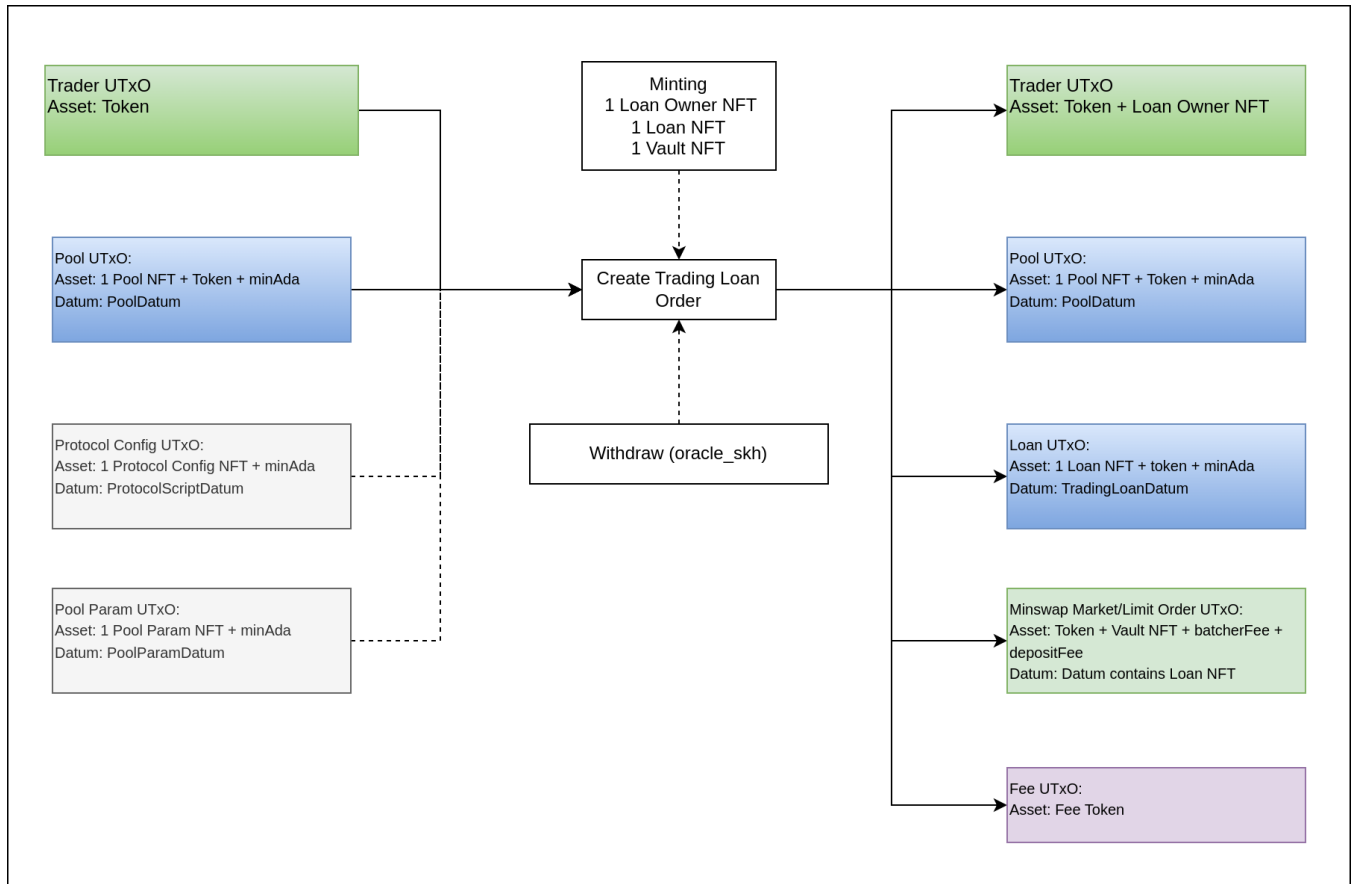
```
PoolDatum {
  total_supply      : Int        -- total tokens deposited by suppliers
  circulating_dtoken : Int        -- outstanding deposit-receipt tokens
  total_borrow      : Int        -- aggregate borrowed amount across all loans
  borrow_apy        : Basis      -- current borrowing interest rate
  undistributed_fee : Int        -- accumulated platform fees not yet withdrawn
  interest_index     : Int        -- cumulative interest multiplier for debt calculation
  interest_time      : Int        -- POSIX timestamp of last interest accrual
  alt_supply_tokens_rate : Pairs<ByteArray, PRational> -- exchange rates for alternative supply tokens
}
```

Pool Param Datum

```
PoolParamDatum {
  collaterals          : Pairs<Token, (LiquidationThreshold, Bool, Basis)>
                        -- accepted collateral tokens with liquidation threshold,
                        -- enabled flag, and liquidator reward rate
  base_rate            : Basis    -- base interest rate
  power_base          : Basis    -- exponential factor for utilization curve
  util_cap            : Basis    -- maximum utilization ratio
  withdrawal_fee       : Int      -- fee charged on supply withdrawals
  loan_fee_rate        : Basis    -- ongoing loan fee rate
  loan_origination_fee_rate: Basis -- one-time fee when creating a loan
  loan_origination_fee_min_amount : Int -- minimum origination fee
  supply_token         : ByteArray -- pool's base lending token
  alt_supply_tokens    : Pairs<ByteArray, Bool> -- alternative yield-bearing tokens
  min_tx_amount       : Int      -- minimum transaction size
  collector_reward     : Int      -- reward for collecting liquidated positions
  platform_liquidation_fee_rate : Basis -- platform's share of liquidation proceeds
}
```

6. Onchain Transaction Flows

6.1 Create Leveraged Position



Actors: Trader

Flow:

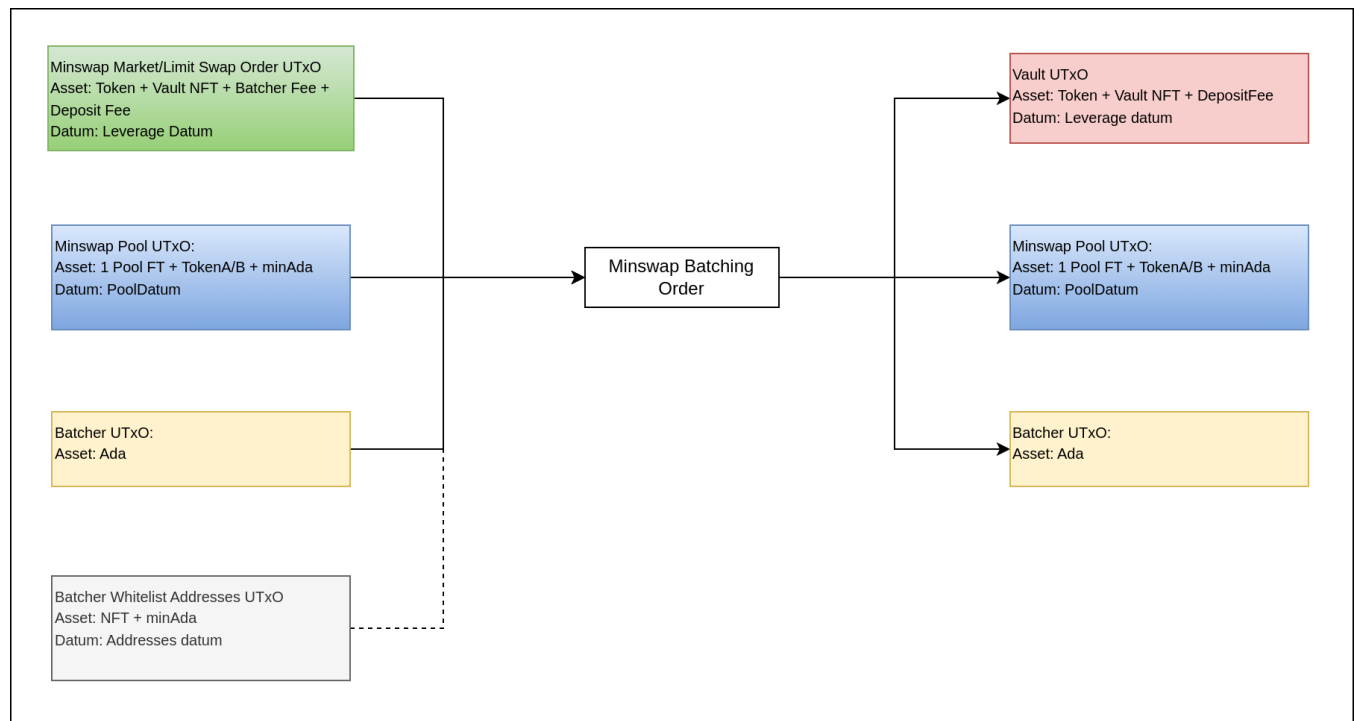
1. Trader submits a transaction consuming their wallet UTxO and the Pool UTxO.
2. The Pool lends the requested amount; borrowed tokens are placed into a Minswap swap order.

- Three NFTs are minted: **Loan NFT** (into Position UTxO), **Loan Owner NFT** (to trader), **Vault NFT** (into Minswap Order UTxO).
- A Platform Fee UTxO is created.

Validation constraints:

- Exactly 1 Loan NFT, 1 Owner NFT, and 1 Vault NFT minted
- Tokens withdrawn from Pool match loan_amount in Loan datum
- Swap order amount + origination fee $\geq$ borrowed amount
- minReceivedAmount of swap order + deposit ensures HF > 1
- Fee amounts match Pool Param configuration

6.2 Minswap Batching (Order Execution)



Actors: Minswap Batcher

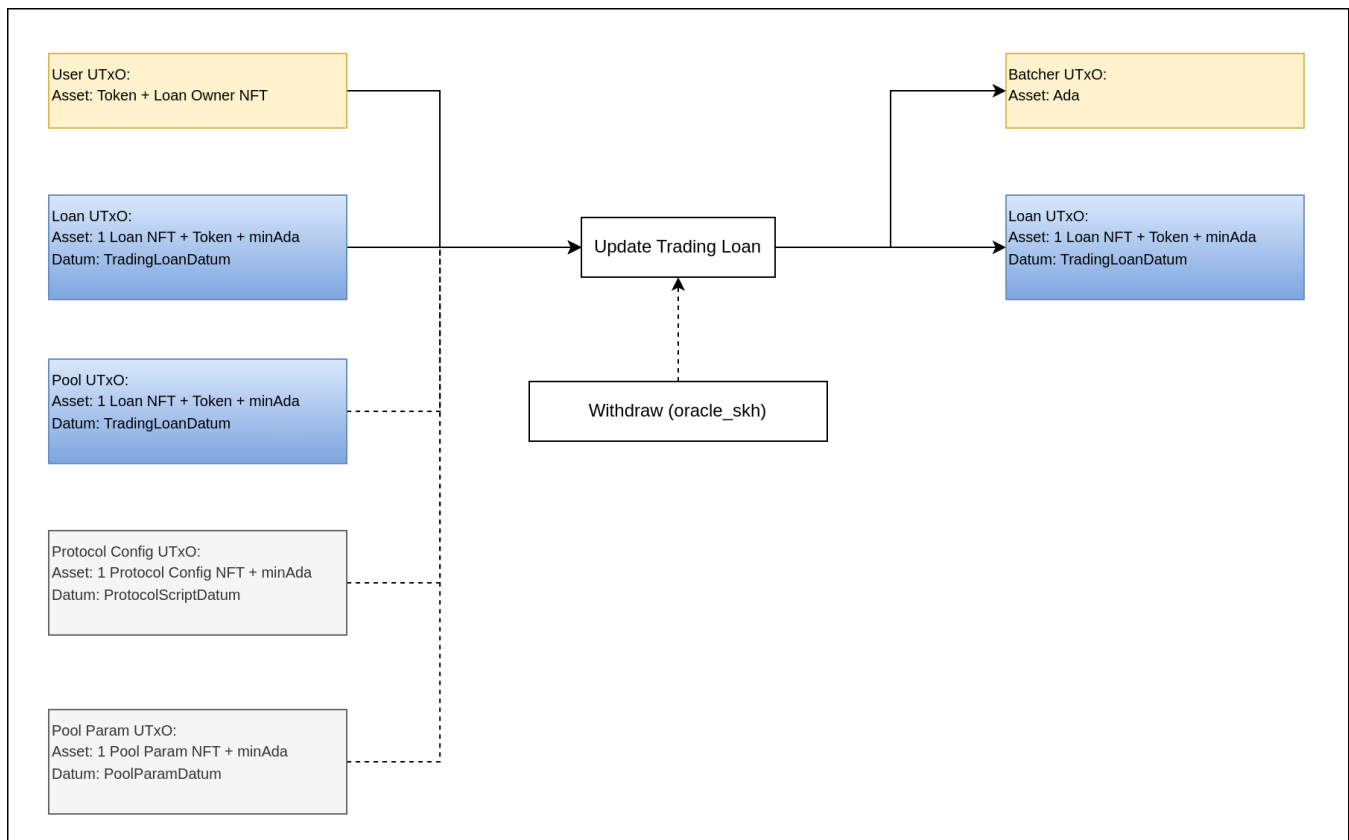
Flow:

- Minswap Batcher matches the swap order against Minswap pool liquidity.
- Swapped tokens (long tokens) are sent to a **Vault UTxO** at the Leverage script address, carrying the Vault NFT and Leverage datum.

Validation constraints:

- Batcher address is whitelisted
- Received tokens $\geq$ minReceivedAmount
- Minswap pool swap formula holds

6.3 Update Trading Loan



Actors: Trader (+ Leverage Admin for swap order cancellation)

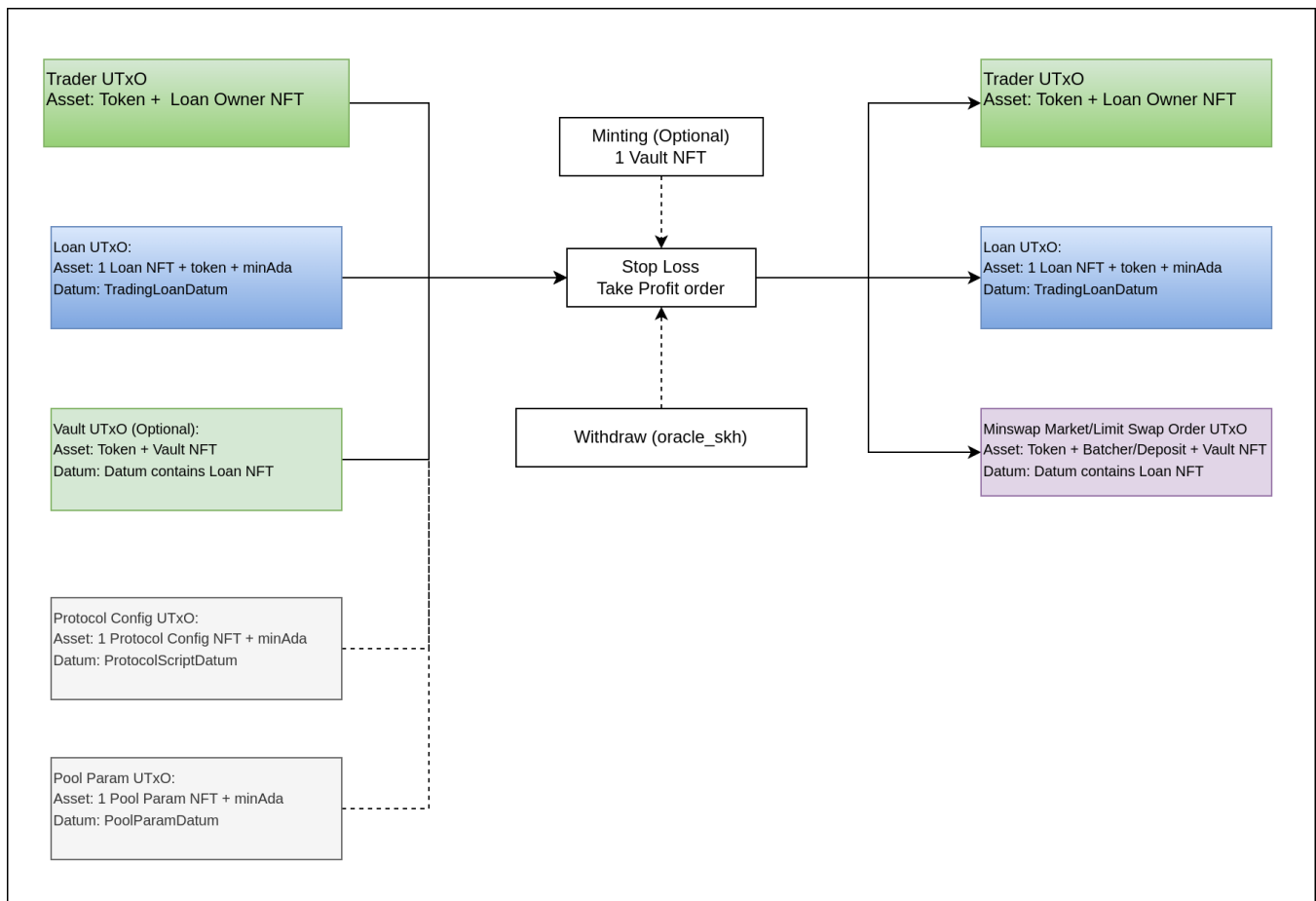
Supports three scenarios:

Scenario	Description
Update Collateral	Deposit/withdraw collateral tokens while maintaining HF > 1
Update Loan Amount (order not yet filled)	Modify the borrow amount, cancel the old Minswap order, create a new one
Update Loan Amount (order already filled)	Modify the borrow amount using the existing Vault UTxO collateral

Validation constraints:

- Loan Owner NFT must be present (as input or reference)
- After update, position must maintain HF > 1
- Pool balance changes must match loan amount delta
- Fee amounts match Pool Param configuration

6.4 Stop Loss / Take Profit



Actors: Trader (order placement) Minswap Batchers (execution) Leverage Bot (collection/distribution)

Phase 1 : Place TP/SL Order:

1. Trader takes collateral from the Vault UTxO and places a swap limit order on Minswap.
2. A new Vault NFT is minted for the TP/SL order; Vault datum received_address is set to:
 - **Pool address** for stop loss (tokens repay the loan)
 - **User address** for take profit (tokens go to the trader)

Phase 2: Minswap Execution: Same as §6.2.

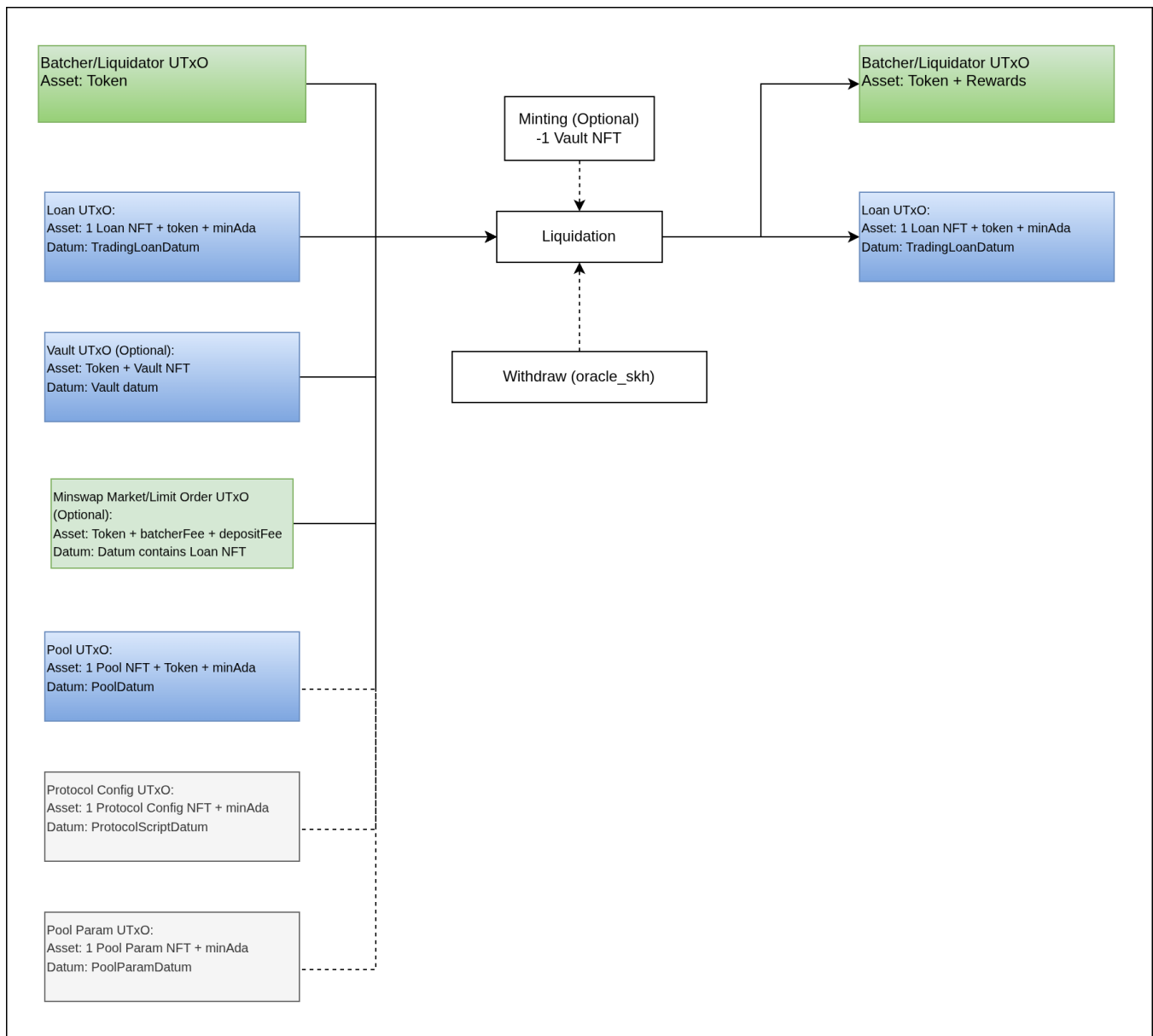
Phase 3a: Bot Collect Stop Loss:

1. Bot collects the filled Vault UTxO and sends repayment tokens to the Pool.
2. Loan datum is updated: loan_amount reduced by repaid amount.
3. Vault NFT is burned.

Phase 3b: Bot Send Profit:

1. Bot collects the filled Vault UTxO and sends profit tokens to the user's designated address.
2. Vault NFT is burned.

6.5 Liquidate Trading Loan



Actors: Liquidator / Bot

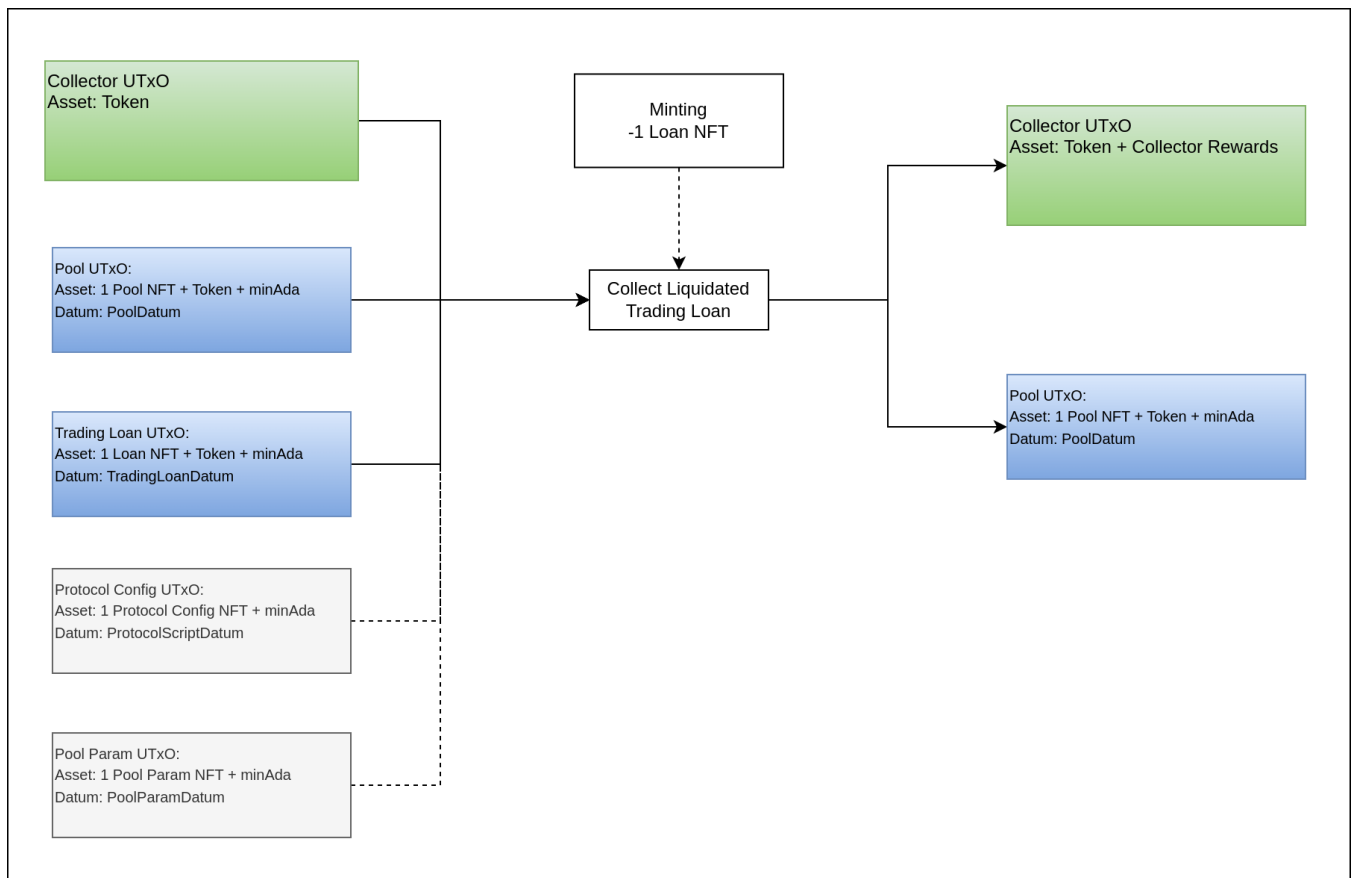
Flow:

1. When $HF \leq 1$, any liquidator can trigger liquidation.
2. The Loan UTxO (and associated Vault/Order UTxOs) are consumed.
3. Remaining tokens are placed into a Collect UTxO awaiting return to the Pool.
4. Liquidator receives a configurable reward.

Validation constraints:

- HF must be ≤ 1 at time of liquidation
- Vault NFT burned if Vault/Order UTxO is consumed
- Reward calculation matches Pool Param configuration

6.6 Collect Liquidated Trading Loan



Actors: Collector Bot

Flow:

1. Bot aggregates one or more liquidated Collect UTXOs.
2. Tokens are returned to the Pool UTXO; Loan NFTs are burned.
3. Collector receives a small reward.

Validation constraints:

- 1 Loan NFT burned per collected position
- Pool receives all tokens from collected positions
- Pool datum updated correctly

7. Health Factor Calculation

The protocol uses the **minReceivedAmount approach** (preferred design) to calculate the Health Factor even before a swap order is filled:

$$HF = (\text{collateral_value} \times \text{liquidation_threshold}) / \text{current_loan_amount}$$

Where:

- **collateral_value** includes: tokens deposited in the Loan UTXO + minReceivedAmount of any pending swap orders + tokens held in filled Vault UTXOs (each valued at its oracle price converted to the borrowed token denomination).
- **current_loan_amount** = $\text{loan_amount} \times (\text{current_interest_index} / \text{initial_interest_index})$: the principal plus accrued interest.
- **liquidation_threshold** is configured per collateral token in Pool Param.

Why minReceivedAmount? Since the actual swap output is unknown until the order is filled, using minReceivedAmount provides a conservative lower bound. This guarantees that once the order is filled (received $\geq$ minReceived), the position's HF remains valid (avoiding immediate post fill liquidation).

Trade off: Traders must deposit slightly more collateral to cover swap fees, slippage, and the liquidation threshold buffer.

8. Risk Management & Liquidation

Risk	Mitigation
Price moves against trader	Automated liquidation when $HF \leq 1$; any user can trigger
Unfilled limit orders	HF calculated on minReceivedAmount; position is still liquidatable
Pool liquidity lock via manipulation	Interest rate model (utilization APY) makes sustained attacks economically unviable
Batcher downtime	Orders remain onchain; positions are safe until matched or cancelled
Smart contract bugs	Multi layer validation (Pool, Loan, Oracle scripts); admin controlled pool parameters

9. Backward Compatibility

The Leverage Trading protocol introduces **new smart contracts** (Leverage Pool, Leverage Loan) alongside the existing Danogo Flexible Lending contracts. The existing Float Pool and Float Loan contracts remain unchanged.

- No existing API fields are removed or altered.
- New fields and endpoints are additive.
- The Pool Param datum structure is compatible with the existing Float Pool Param, enabling future unification.

This document provides a high level architectural overview of the Danogo Leverage Trading Protocol. For detailed UTXO diagrams, validator logic, and redeemer specifications, refer to the internal TDD on Confluence.